

DNS Resolver & Query Analyzer with Performance Comparison

Submitted By: Amna Asim Khan

ID: 23K-0859

Section: BAI-6A

FAST School of Computing

National University of Computer & Emerging Sciences

1. Motivation

*The Domain Name System (DNS) is one of the most essential components of modern computer networks and the Internet. Every website visited by users relies on DNS services to translate human-readable domain names into machine-readable IP addresses. Despite being a core networking service, many students only study DNS theoretically and do not gain practical exposure to how DNS resolution actually works internally.

The motivation behind this project is to develop a practical DNS Resolver and Query Analyzer that demonstrates the complete DNS querying process while also analyzing the performance of different DNS servers. The project aims to bridge the gap between theoretical concepts and real-world implementation by allowing users to observe DNS communication, response times, DNS records, and packet-level interactions.

Another motivation for selecting this project is the increasing importance of network performance optimization and DNS efficiency in modern networking environments. Public DNS services such as Google DNS and Cloudflare DNS are widely used because of their speed and reliability. Comparing the performance of these DNS servers provides useful insights into network latency and server responsiveness.

This project also provides an opportunity to strengthen practical skills in Python programming, network protocol analysis, packet capturing using Wireshark, and performance monitoring. The implementation combines networking concepts with software development, making it academically valuable and technically practical for Computer Networks coursework.*

2. Overview

The Domain Name System (DNS) is responsible for translating domain names into IP addresses that can be understood by network devices. Whenever a user accesses a website such as google.com or youtube.com, the DNS server resolves the domain name into an IP address so that communication between the client and the destination server can occur.

The proposed project is a Python-based DNS Resolver and Query Analyzer that allows users to send DNS queries to multiple public DNS servers and analyze their responses. The system retrieves various DNS records, measures response times, compares DNS server performance, and logs DNS query information for analysis.

The resolver will use the Python dnspython library to send DNS queries and retrieve responses. Users will be able to choose the type of DNS record they want to query, including A, AAAA, MX, NS, and CNAME records.

The project also includes Wireshark-based packet analysis to observe DNS traffic at the packet level. DNS packets generated by the application will be captured and analyzed to demonstrate the DNS query and response process.

The final system will provide both educational and analytical functionality by demonstrating how DNS works internally and how different DNS servers respond under various conditions.

2.1 Significance of the Project

This project is significant because DNS is one of the most critical protocols in computer networking. Understanding DNS resolution is essential for network engineers, cybersecurity professionals, and system administrators.

The project has the following significance:

- Provides practical implementation of DNS concepts studied in Computer Networks.
- Demonstrates client-server communication using real DNS servers.
- Helps understand DNS packet structure and communication flow.
- Measures DNS response times to analyze network latency.
- Allows comparison between multiple DNS service providers.
- Introduces packet-level analysis using Wireshark.
- Enhances programming and networking integration skills.

Academically, the project strengthens understanding of DNS protocol operations, packet transmission, and network performance analysis.

Practically, the project demonstrates how DNS performance affects overall network responsiveness and user experience.

2.2 Description of the Project

The DNS Resolver and Query Analyzer is a command-line Python application that sends DNS queries to public DNS servers and analyzes their responses.

The system accepts a domain name as input from the user and allows the user to select a DNS record type. The resolver then sends the DNS query to one or more DNS servers and retrieves the requested records.

The system supports the following DNS record types:

- A Record (IPv4 Address)
- AAAA Record (IPv6 Address)
- MX Record (Mail Server)
- NS Record (Name Server)
- CNAME Record (Canonical Name)

After retrieving the response, the application displays:

- Resolved IP address or DNS record value
- DNS server used
- Query response time
- TTL (Time-To-Live)
- Record type

The project also compares the performance of multiple public DNS servers including:

- Google DNS (8.8.8.8)
- Cloudflare DNS (1.1.1.1)
- OpenDNS (208.67.222.222)

The application measures and compares the response times of these DNS servers and identifies the fastest responding server.

Additionally, all query results are saved into a log file containing:

- Domain name
- Query timestamp
- DNS server used
- Record type
- Response time

- Query result

Wireshark is used to capture DNS packets generated by the resolver. Packet analysis helps visualize DNS queries and responses and demonstrates DNS communication at the network layer.

The system also includes error handling mechanisms for invalid domain names, unavailable DNS servers, and network connectivity failures.

2.3 Background of the Project

DNS was introduced to simplify Internet communication by replacing numerical IP addresses with easy-to-remember domain names. The protocol operates in a hierarchical and distributed manner, making it scalable and reliable for global Internet communication.

A DNS resolver is responsible for querying DNS servers and retrieving the requested records. DNS queries typically use UDP port 53 for communication. DNS records contain various types of information such as IP addresses, mail servers, and name servers.

Public DNS providers such as Google DNS and Cloudflare DNS offer high-speed and secure DNS services for Internet users worldwide.

Several tools already exist for DNS querying, such as nslookup and dig, but this project focuses on implementing similar functionality programmatically using Python.

The following technologies and resources were studied during the project:

- DNS protocol and RFC 1035 documentation
- Python dnspython library
- Wireshark DNS packet analysis
- Network latency and response time analysis
- DNS record structures and TTL values

The project combines practical networking concepts with software implementation to provide an educational DNS analysis platform.

2.4 Project Category

This project falls under the following category:

Product-Based Project

The project produces a functional software application capable of performing DNS resolution and DNS response analysis.

Research-Based Elements

The project also includes analytical and performance comparison features that evaluate DNS server responsiveness and DNS behavior.

3. Features / Scope / Modules

The following features are implemented in the proposed project:

3.1 Domain Name Resolution

The system allows users to input a domain name and retrieve its corresponding IP address using DNS queries.

Example:

- google.com → 142.250.x.x
- youtube.com → 172.x.x.x

This feature demonstrates the core functionality of DNS resolution.

3.2 Multiple DNS Record Support

The resolver supports multiple DNS record types including:

- A Record
- AAAA Record
- MX Record
- NS Record
- CNAME Record

This feature allows users to analyze different types of DNS information associated with a domain.

3.3 DNS Server Performance Comparison

The system queries multiple DNS servers and compares their response times.

Supported DNS Servers:

- Google DNS (8.8.8.8)
- Cloudflare DNS (1.1.1.1)
- OpenDNS (208.67.222.222)

The application identifies the fastest responding DNS server.

3.4 DNS Response Time Measurement

The application measures the time taken by each DNS server to respond to a query.

This feature helps analyze network latency and DNS performance.

3.5 TTL (Time-To-Live) Analysis

The resolver extracts and displays the TTL value returned by DNS records.

TTL indicates how long a DNS response can remain cached before refreshing.

3.6 DNS Query Logging

All DNS query results are stored in a log file for future analysis.

The log file contains:

- Domain name
 - Record type
 - DNS server
 - Response time
 - Query result
 - Timestamp
-

3.7 Packet Capture and Analysis

DNS packets generated by the resolver are captured using Wireshark.

This feature allows visualization and analysis of:

- DNS Query Packets
- DNS Response Packets
- DNS Headers

- UDP Communication
-

3.8 Error Handling

The system gracefully handles errors such as:

- Invalid domain names
 - DNS server timeouts
 - Internet connectivity failures
 - Unsupported record types
-

3.9 Simple Performance Visualization

The project includes basic response-time visualization using Python graphs.

This allows users to compare DNS server performance visually.

4. Project Planning

The project was planned over a period of five weeks.

Week	Tasks
Week 1	Research DNS protocol, install Python, dnspython, and Wireshark
Week 2	Develop basic DNS resolver and implement A record querying
Week 3	Add support for MX, NS, AAAA, and CNAME records
Week 4	Implement DNS server comparison and response-time measurement
Week 5	Add query logging, packet analysis, testing, and documentation

Individual Responsibilities

Since the project is being completed individually, all project tasks including research, implementation, testing, packet analysis, documentation, and presentation preparation are handled by the same student.

5. Project Feasibility

5.1 Technical Feasibility

The project is technically feasible because all required technologies are easily available and supported on modern systems.

The project uses:

- Python programming language
- dnspython library
- Wireshark packet analyzer

These technologies are lightweight, open-source, and compatible with standard operating systems.

The project does not require specialized hardware or advanced infrastructure.

Potential technical challenges include:

- DNS server timeouts
- Internet connectivity issues
- Packet capture permissions

However, these issues can be managed through proper error handling and testing.

5.2 Economic Feasibility

The project is economically feasible because it requires minimal financial resources.

Required software tools are free and open-source:

- Python
- dnspython
- Wireshark

The project can be developed on a standard personal computer without additional hardware costs.

Therefore, development and operational costs are very low.

5.3 Schedule Feasibility

The project is feasible within the given academic timeline.

The implementation is divided into manageable milestones over five weeks.

Since the project scope is moderate and technologies are lightweight, the project can be completed successfully within the allocated semester duration.

6. Hardware and Software Requirements

Hardware Requirements

- Personal Computer or Laptop
 - Minimum 4 GB RAM
 - Internet Connection
 - Intel i3 Processor or Equivalent
-

Software Requirements

Software	Purpose
Python 3.x	Main programming language
dnspython Library	DNS query handling
Wireshark	Packet capture and analysis
VS Code / PyCharm	Code editor
Windows / Linux OS	Development environment

7. Diagrammatic Representation of the Overall System

System Workflow

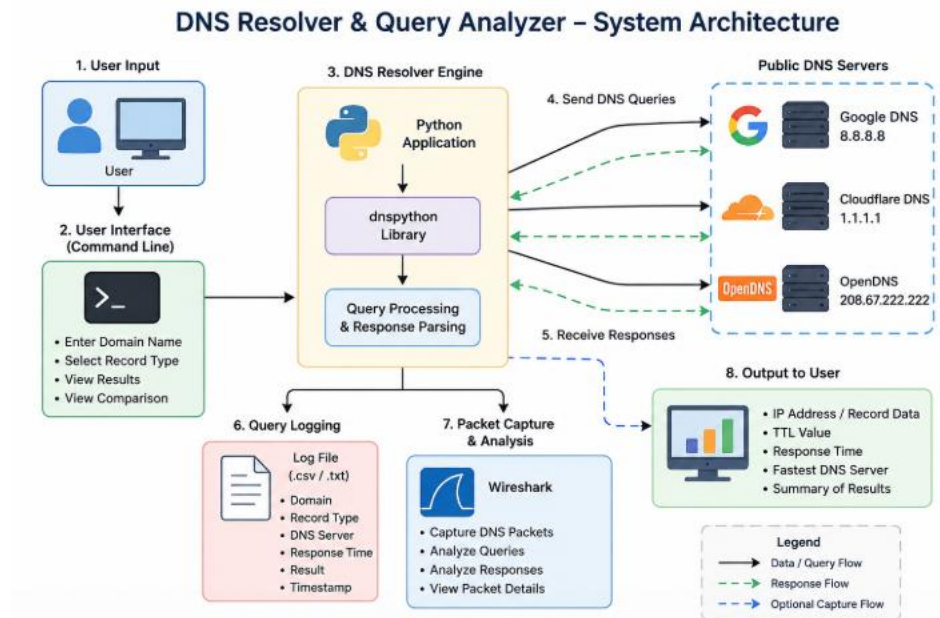


Figure 1. Overall workflow of the DNS Resolver and Query Analyzer system.

8. References

- [1] P. Mockapetris, "Domain Names - Implementation and Specification," RFC 1035, November 1987.
- [2] James F. Kurose and Keith W. Ross, *Computer Networking: A Top-Down Approach*, Pearson Education.
- [3] dnspython Documentation. Available: <https://www.dnspython.org>
- [4] Wireshark Network Protocol Analyzer. Available: <https://www.wireshark.org>
- [5] Cisco Networking Academy, "Introduction to DNS and Networking Concepts."
- [6] Python Software Foundation, "Python Documentation." Available: <https://docs.python.org>